Faculty of Business Informatics

Scientific Team Project

Test Driven Development of Big Data Application using Microservices Architecture

Team:
Apoorva Chandrashekar
Nikhilkumar Italiya
Priyanka Byahatti
Suhas Shantharam

Supervisor: Daniel Staegemann
Reviewer: Prof. Dr. rer. nat. habil. Klaus Turowski
Faculty of Business Informatics
Otto-von-Guericke University
Universitätsplatz 2
39106 Magdeburg, Germany

Table of Contents

Project Objective **3**

Introduction **3**

Implementation **4**

Technology Stack 4

Data Sources 4

Use case implementation 4

Visualizer 4

Movie tags predictor 6

Recommendation Service 8

Dashboard 10

Building Docker containers 11

Project workflow using GitHub Actions 11

Test Driven Development **11**

Unit Testing 11

Integration testing 12

Using the Application **13**

Limitations **15**

Lessons Learned **15**

Conclusion **15**

Future Work **16**

References **17**

Project Objective

The goal of the project is to leverage Test-Driven Development strategy in implementing Big Data applications using Microservices architecture.

To illustrate this, we have chosen to build an application that uses various microservices internally that are independent of each other. We have built 3 different services in this application namely movie tag predictor, visualizer, and a recommendation service. We have developed this application by using the Test-Driven approach, where the automated test scripts are written first followed by the development process.

Introduction

Microservices is a type of an architecture or framework to build an application. In this type of architecture, the blueprint or initial plan of an app is broken into modules. These modules are loosely coupled with each other and are also independent in nature. As such, it offers many benefits when compared to the monolithic style of building apps or services like increased flexibility in terms of design, easily scalable, high availability in case of cloud applications and so on.

In today's day and age, the abundance of data has made it very important the approach and manner in which we utilise the data present. Data can come from a variety of sources and can be of different sizes and structure. If the data generated is handled in an efficient manner the insights and information that can be obtained is higher [1]. Dealing with huge amounts of data using the traditional monolithic architectures might be challenging due to scaling issues and service outages in the applications since the entire app is run on one single code base. However, with the use of microservice architecture this can be overcome since it gives room to a more accessible code base which is easier to edit and maintain in the longer run. It also helps build services with automated integration testing, CI/CD and agile development. Although not prevalent, study has shown that microservice architecture for big data with a test driven approach having good test coverage has the possibility to improve the trust placed on such kinds of applications and its acceptance in the user community [2].

Implementation

Technology Stack

We are using Python and Javascript to build our microservices. To store all the data we have made use of PostgresQL Database. For the frontend UI, we have used HTML and CSS to design simple pages. To containerize the application, we have used Docker. For testing, we have used the Pytest framework from Python's libraries. The entire project workflow is set up using Github Actions.

Data Sources

We are sourcing data for the project from different movie sources such as IMDB, Kaggle and Movie Lens. The IMDB dataset contains different tables such as ratings, titles, movies. The movie reviews data has been sourced from Kaggle. Movie Lens Database also has been used to leverage ratings, tags, movies etc.

IMDB Dataset: This publicly available data is sourced from the IMDB website which is hosted on https://datasets.imdbws.com/ . The dataset contains title ratings in tabular format:

- tconst - contains a unique id to identify the title of each movie .
- averageRating - it contains the weighted average of the rating given by users
- numVotes - number of votes the title of the movie has received.

Movie Lens Dataset: This data is sourced from the Movie lens website which is hosted on: http://files.grouplens.org/datasets/movielens/ml-latest-small.zip. The dataset contains ratings of movies from MovieLens website which is a movie recommendation service. It contains ratings of 9742 movies along with its ratings and tags.

The dataset contains 3 files which we make use of all in tabular format:

Movies data:

- Movie ID: A unique id to identify the title of each movie.
- Title: Contains the title of the movie.
- Genres: Contains information regarding which genres the movie belongs to.

Links data:

- Movie ID: Same value as in Movie file.
- IMDB ID: Contains the corresponding IMDB id of the movie.

Ratings data

- User Id: Unique ID to identify the user who has provided feedback in the form of rating on a scale of 1 to 5.
- Movie ID: Same value as in movies file.
- Rating: The ratings provided by the user.

Kaggle Dataset: The data used here is "A Corpus of Movie Plot Synopsis with Tags (MPST)" and is accessed on the popular Data Science platform Kaggle. The data consists of 14,828 movies and 6 attributes describing the movies.

Synopsis data

- Imdb_id: IMDB id of the movie
- title: Title of the move
- Plot_synopsis: Plot Synopsis of the move
- Tags: Tags assigned to the move
- Split: Position of the movie in the standard data split
- Synopsis_sources: From where the plot synopsis was collected

Use case implementation

1. Visualizer

Data Source: IMDB, MovieLens

This service shows visualisations of various aspects of movies from the IMDB dataset such as highest rated movies, lowest rated movies, movies made per year, movies made by genre. The Visualiser service contains 3 Microservices: Data Collector, Database and Web Page.

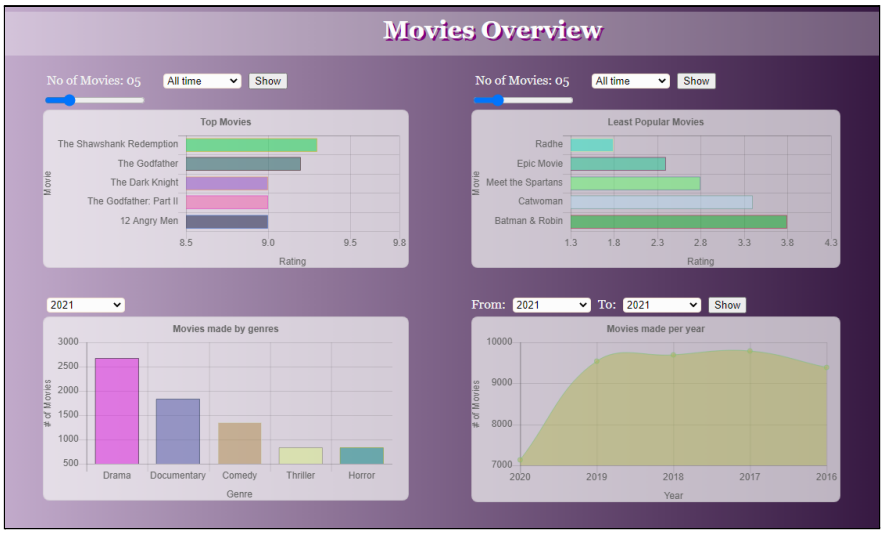

Fig.1 Visualizer service

- **Data Collector**: Data Collector collects the data from IMDB website and then sends it to the Database after cleaning the data. Apart from IMDB movie dataset, this service also uses the Movielens dataset containing ratings given by users to the movie. The Microservice is made using python and has 2 API endpoints.

API endpoints	**Description**
/api/load-data/{password}	To load the IMDB data and send it to database
/api/check-status	Check the status of Microservice

- **Database**: In order to avoid downloading data every time a user opens a webpage, a database is used to store the cleaned movie data. And to be accessed by web page when needed. Postgresql is used for this Microservice.
- **Webpage**: This Microservice contains a web page where users will interact. It is developed in Javascript (Nodejs). The microservice is connected with the database and takes the data from the database as per requirement. The landing page shows 4 different charts. Data shown in the charts can be modified based on different parameters given to the charts. For example in the Highest rated movies chart, users can change year and number of movies using a slider to display top rated movies of his/her choice.

Apart from this, users can also get more details about any particular movie shown in the chart by clicking on the bar related to that particular movie. Upon clicking, a new webpage will open with few details of that movie such as synopsis , tags (from tags predictor service) and recommendations (from movie recommender service). In order to get the synopsis, we are using the imdb id from the previous page to scrape the synopsis from the IMDB website.

Fig. 2 Collective view of Tags prediction & recommendation for a movie

Upon starting the microservice, it runs integration tests. First of all it checks whether all the required microservices are online. If this test passes, it checks the database for a copy of older data. If old data is found, the webpage microservice opens the port for users showing visualisations from old data and sends api requests in the background to Data Collector microservice to download new data and updates the visualisations as soon as the new data is stored in the database. If old data is not found in the database, then it sends API requests to the Data Collector and waits until the data is stored to the database before opening port for users.

2. Movie Tags Predictor

Data Source: Kaggle

The data used in Movie Tags Predictor service is accessed from Kaggle website. It contains approximately 14K+ movies and different attributes explaining the movies. Plot_Synopsis is

the primary independent attribute, and Tags is the dependent target variable. The plot synopsis data is used for training the Naive Bayes Classifier.

The idea behind this service is to collect the plot synopsis of any movie and return hash tags that summarise the overall theme of the movie. Below is the simple UI of the service. The user can input any text (in this case the plot of any movie) into the text area and hit the submit button to receive hash tags representing the movie below [3].

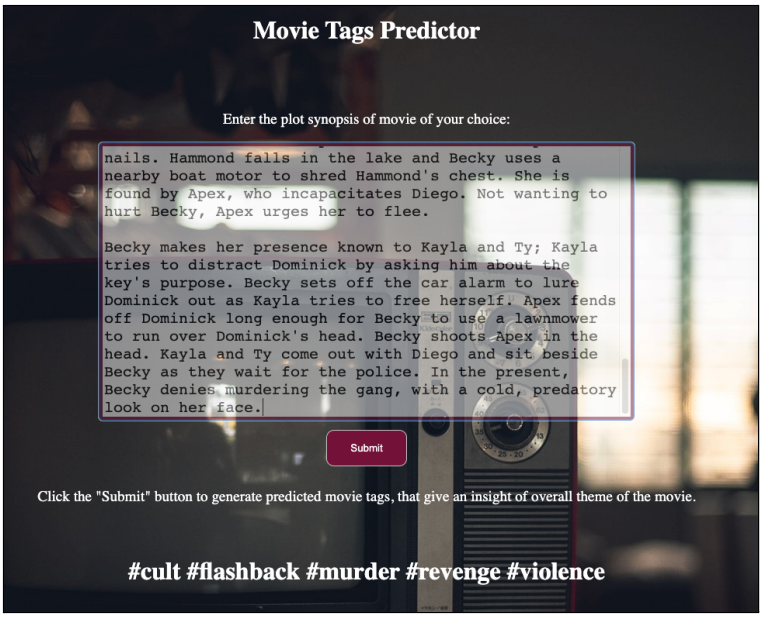

Fig. 3 Movie Tags Predictor service

This is essentially one of the microservices applications which can be used independently of any other service. The service in turn is divided into several other services such as processor, predictor and webpage. The latter services communicate with each other using REST Api's. When the service is started, all three containers (processor, predictor and web page go live). During this process, the processor downloads the labelled data (containing several movie variables with 'tags' as a target variable) from Kaggle and then performs some data preprocessing. Predictor gets the processed data from the processor using API and builds a Machine Learning model on this data.

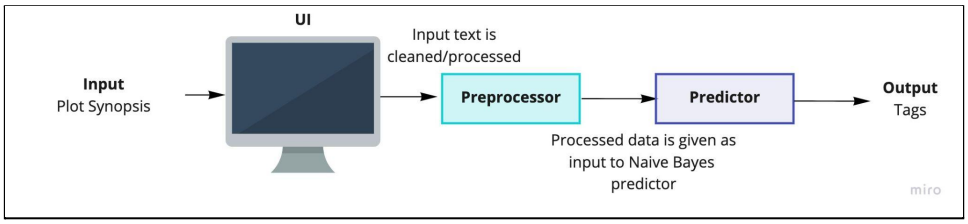

Fig. 4 Architecture of Movie Tags Predictor

- **Webpage service**: This contains UI files such as HTML, CSS files and also contains the core Flask app logic, where we have initiated the flask app on localhost 3311 port.

- **Preprocessor**: This service is programmed to source the labelled data from Kaggle, and then perform some text processing techniques on this main input variable that is 'plot_synopsis' such as lemmatization, stop words removal and stemming using nltk package from Python. Once the cleaned synopsis is clean and ready, it is loaded into Postgres tables for storage purposes.

- **Predictor**: At the first instance when an application is live, this service builds an ML model on top of the processed data received from the preprocessor, and saves the model. When it receives test data i.e every time a user inputs a new synopsis, it gets this data from preprocessor and gives it as an input to Naive bayes model and outputs the generated prediction.

3. Recommendation Service

Data Source: MovieLens

Movie Recommender application recommends the user the next movie to watch when he/she provides the name of the movie that is already watched as an input. Movie Recommender Service is made of 4 Microservices: Data Collector, Database, Recommender, Webpage.

- **Data Collector**: This Microservice developed in python, downloads the data from Movielens and sends it to the database after basic cleaning. This Microservice has 2 API endpoints.

API endpoints	**Description**
/api/load-data/{password}	To download the data and send it to database
/api/check-status	Check the status of Microservice

- **Database**: Database Microservice stores the data from Data Collector. It uses Postgresql.

- **Recommender**: The main function of recommender microservice is to give recommendations of any movie requested by the user. The recommendations are provided using collaborative filtering methods. The Movielens dataset contains information about movies and ratings given by each user. With help of this information, a matrix is created and the recommendations are provided using k nearest neighbour method. This microservice is developed in python and has 3 API endpoints:

API endpoints	**Description**
/get-recommendation/{movie name}	To get the recommendations using Movie name
/get-recommendation/id/{imdb_id}	To get the recommendations using IMDB ID of a movie
/api/check-status	Check the status of Microservice

- **Webpage**: User interaction happens with the Webpage Microservice. Developed using Javascript, this microservice provides a User Interface for users to get the recommendations of any movie. The microservice takes into account all the movies stored in the database which is then used for auto-completion. When the user hits on the recommend button, it sends an API request to Recommender Microservice and returns all the received recommendations. The results can be removed using the reset button.

Before opening the port for users, it checks whether data is available in the database. If old data is there, the port is opened and a request for downloading fresh data is sent to the Data Collector.

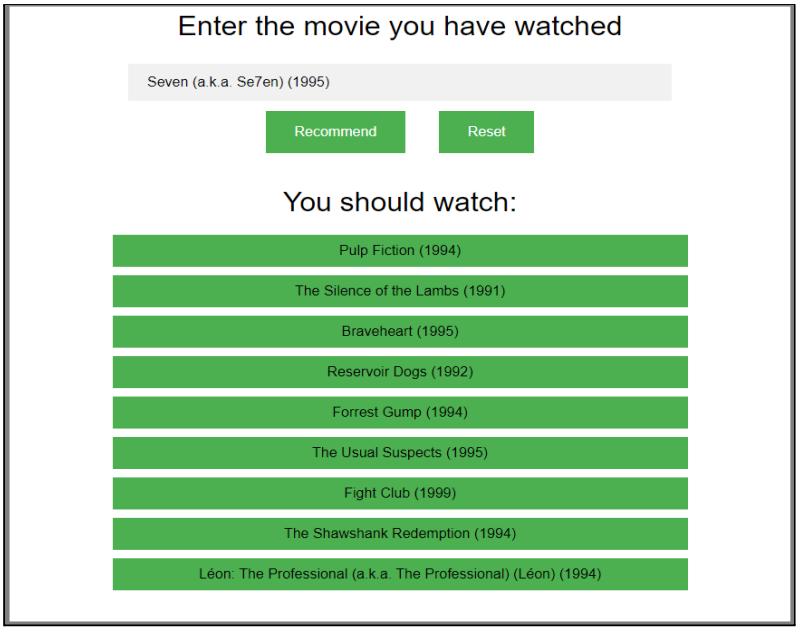

Fig. 5 Recommendation service

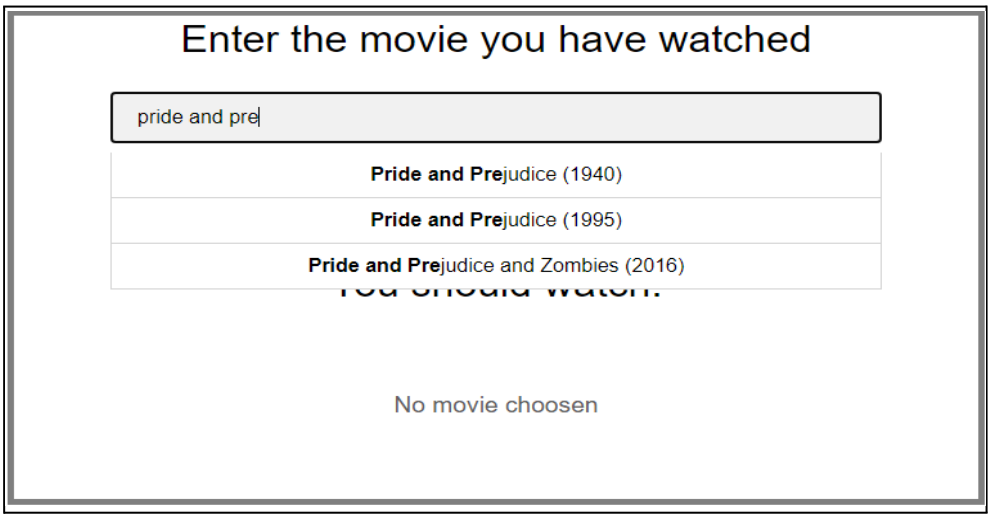

Fig. 6 Movie search auto completion feature

4. Dashboard

This microservice is independent of all the other services. It is built to get the overview of all the Microservices. It uses python and Javascript. The webpage shows all the microservices along with its status, whether it is online or offline. Each microservices can also be manually checked using the check button. The microservice is also designed to automatically re-check the status of every microservices after a specific time interval. Currently it is set to 5 Minutes. Apart from this, the microservice will also update the data every 24 hours in the database by sending a request to the respected microservice. All the microservices (Webpages) that can be accessed by users are hyperlinked and can be accessed from the dashboard.

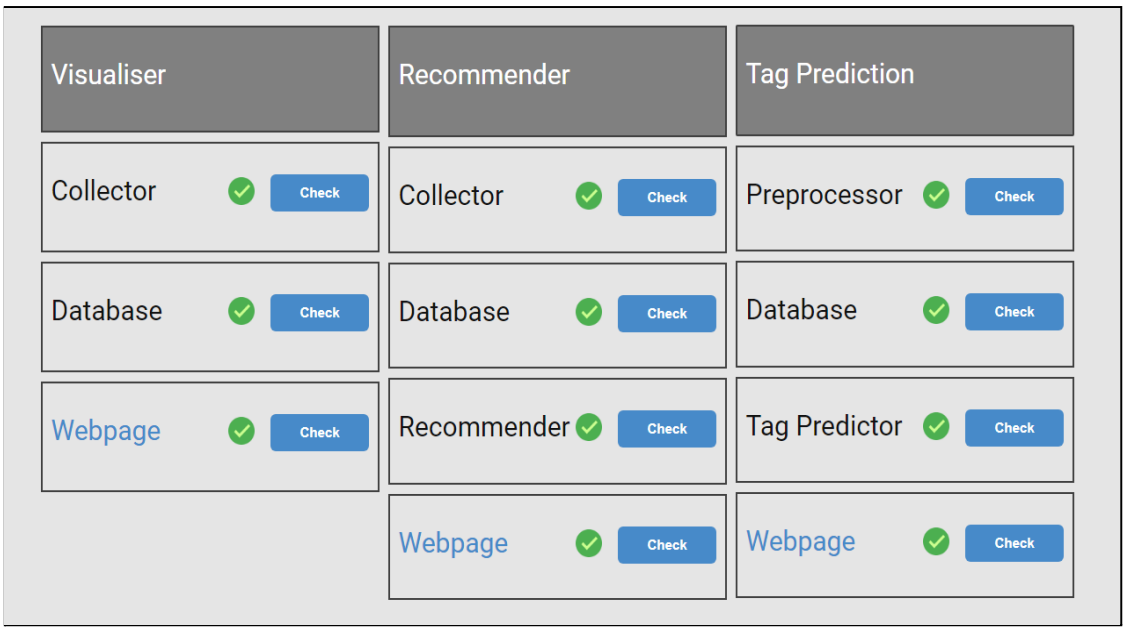

Fig. 7 Dashboard status of services

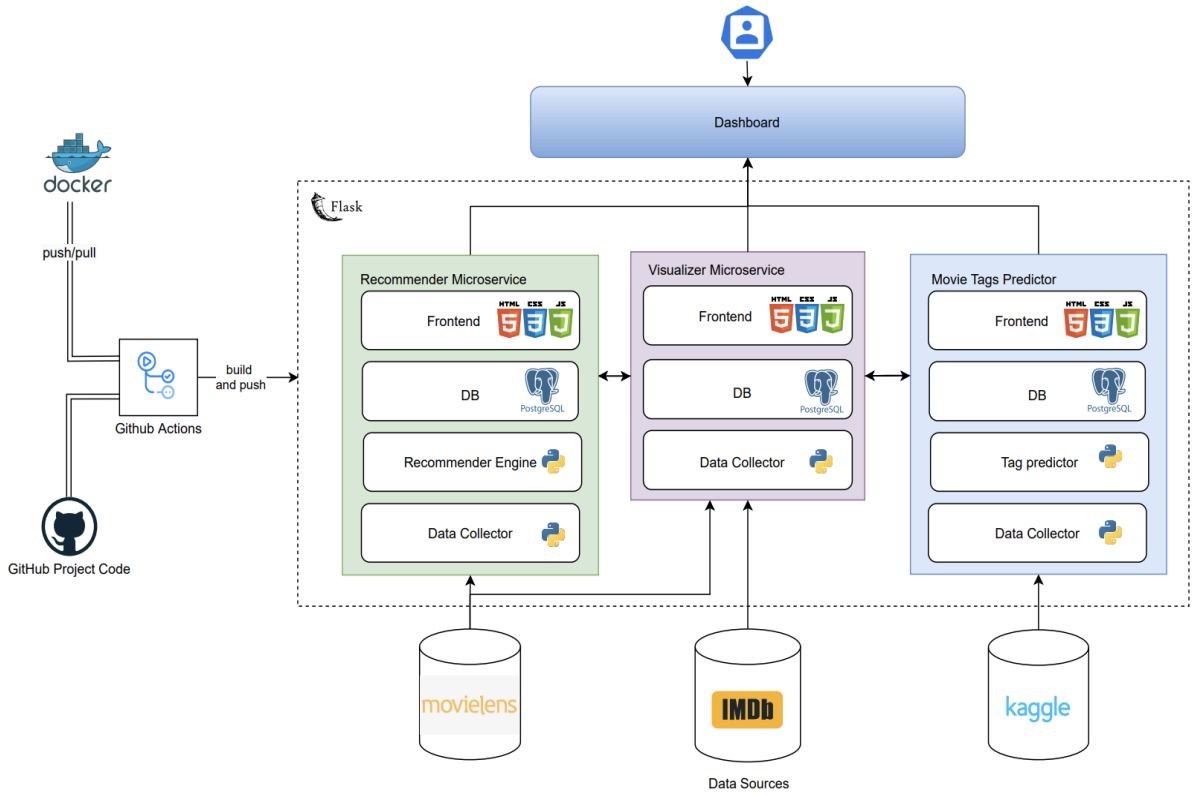

Fig. 7 Microservice architecture of Project Movies

Building Docker containers

Docker is a containerization platform used to build, deploy and manage containerized applications. Docker provides an isolated environment for applications with the operating system and dependencies required to run that application which makes it easier to deploy the application in any environment [4].

In this project, we have built separate containers for each microservices. This will allow independent development and scaling for any particular microservice. Each microservice contains predefined API paths which enables it to perform actions such as check status of microservice and run unit tests.

Replaceable Containers

Most of the data transfer and connection between each microservices is done using API endpoints. Therefore, any container can be replaced by modifying the source code in that particular container and keeping the API endpoints the same. New container should have an obligatory API endpoint "/check-status" to check if the service is running properly.

Project workflow using GitHub Actions

We used GitHub as a version control system. GitHub provides CI/CD tool GitHub Actions which helps to automate the development workflow. Upon pushing new changes to the repository, it automatically runs test scripts for all the microservices and builds the containers

The below image shows the CI/CD workflow output of visualizer use case

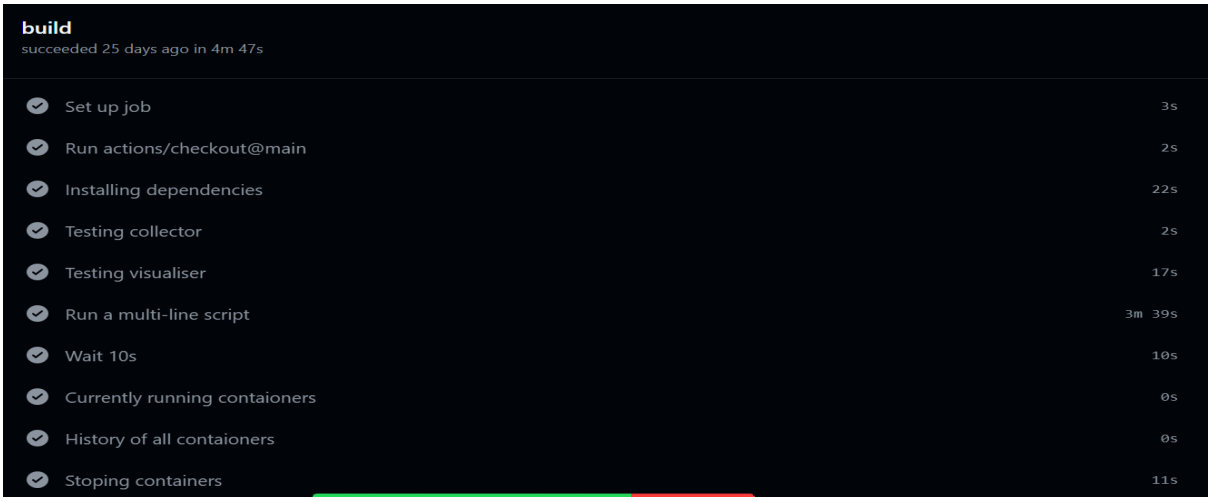

Fig. 9 Github Actions workflow for Visualizer service

Pipeline for Visualiser Service:

- Install required dependencies eg. python, node, pytest etc.
- Run all the unit tests for Data Collector service
- Run all the unit tests for Visualiser service
- Build the image for all the containers and run it
- Wait for the containers to be online
- List all the running containers
- List all the running and stopped(if any) containers

Test Driven Development

Test driven development is a testing approach in which test scenarios are formed and tested first to validate the code, and then the development proceeds depending on test pass/fail. The primary goal of this technique is to modify or write new code only if the tests fail. This prevents duplication of test scripts. For our project we used the popular Python framework "Pytest" to write our test scripts.

Approaches:

1. Developing tests as we go

   In this approach, the unit tests can be developed as the developer implements the application further.

   - Pros: Flexibility in this approach helps the developer to decide for which cases he needs to write the test case first and for which he needs to write the code first.
   - Cons: Requires a culture change in the mindset of the developer who is used to writing code first and then test cases. This also can act as a disadvantage of the entire TDD approach.

2. Develop tests all at once

   In this approach, all the unit tests for the entire Application are developed first and then the functions passing the tests are implemented. It is required to understand and modularize the functionality of the entire application to apply this approach. We used this approach for building our Microservices.

- Pros: Makes it easier to collaborate, as the entire architecture is planned out ahead. Can be confident with the developed code as it implements the whole application functionality if all the test cases pass successfully. This approach is best for small applications where the number of test cases to be implemented is relatively small
- Cons: Implementing large applications using this approach will get complex. The actual implementation of the app itself gets delayed as time is consumed in designing all the test cases in the beginning of the project. This might be crucial in cases where implementation of the application has a deadline.

3. Develop milestone tests first and then develop tests as we go

   Instead of developing all the tests for the entire application, only some tests which could be marked as milestones of development are developed first. Then between these milestones, tests can be developed as the developer further implements the application.

   - Pros: Reduces the time required to develop all the tests in the beginning of the project like the second approach. Suitable for developing huge applications.
   - Cons: Difficult to determine which functionalities of the app acts as the major part of implementation so that the test cases can be written for it at the start.

1. Unit Testing

Each of our microservices contain testing scripts, which validate each containers' results before these containers are executed.

To illustrate, below is a simple test function that tests the functionality of the function "decontracted(phrase)". This function converts each contraction of phrase to its expanded, original form thereby helping with text standardisation. The "test_decontracted()" is a test function that validates this function.

```
def test_decontracted():
    phrase = "I won't do it"
    phrase = decontracted(phrase)
    assert phrase == 'I will not do it', 'decontracted function
not working'
```

Fig. 10 Example of unit test for text processing

Following are few more examples of test coverage:

- To check whether the correct data is downloaded from the link the provided
- To check whether the link is up and running and has no issues from the server on which it is hosted
- To check whether the connection to the database is established
- To check whether the tables are created in the database to store the downloaded data
- To check whether the data is present in the tables created

2. Integration testing

Integration tests are implemented in web page Microservice in all the use cases. The webpage microservice in any use case will start after all the other Microservice related to that use case have started. Therefore, it is important to have integration tests on webpage Microservice. Integration tests will check if all the Microservice required for web page Microservice to work properly are running. Then using dummy data, the functionality will be tested. If the results from these tests match the expectations, then the port for that use-case is opened for the users.

For example: The webpage Microservice in recommender use case requires Database, Collector and Recommender Microservices to be up and running. So first the webpage Microservice will check status for these Microservices. If all of them are running, then it will send a dummy Movie Name and try to get movie recommendations as a response. If such recommendations are received, the service is ready for users and the port for users will be opened.

Using the Application

1. Cloning the repository

The main repository contains references as submodules to the repositories of all 3 implemented use-cases and an additional Microservice “Dashboard” to check status of all the Microservices. It is required to have all the files from the submodules locally available.

1.1. To clone the repository and submodules:

```
git clone --recursive

https://github.com/iamitaliya/big_data_microservices.git
```

1.2. Host system needs to have Docker installed. Next, docker image can be built and run by following command from project directory using terminal:

```
docker-compose up -d
```

1.3. The application will be hosted on the localhost:5555

Since all the services Include dockerfile, no dependencies except Docker need to be installed from the host machine.

2. File Structure

```
Big Data Microservices
|-- Dashboard
|    |-- Dockerfile
|    `-- ...
|-- Visualiser
|    |-- Database
|    |-- Collector
|    |    |-- Dockerfile
|    |    `-- ...
|    |-- Webpage
|    |    |-- Dockerfile
|    |    `-- ...
|    `-- docker-compose.yml(Visualiser)
|-- Recommender
|    |-- Database
|    |-- Collector
|    |    |-- Dockerfile
|    |    `-- ...
|    |-- Recommender
|    |    |-- Dockerfile
|    |    `-- ...
|    |-- Webpage
|    |    |-- Dockerfile
|    |    `-- ...
|    `-- docker-compose.yml (Recommender)
|-- Tags Predictor
|    |-- Database
|    |-- Preprocessor
|    |    |-- Dockerfile
|    |    `-- ...
|    |-- Predictor
|    |    |-- Dockerfile
|    |    `-- ...
|    |-- Webpage
|    |    |-- Dockerfile
|    |    `-- ...
|    `-- docker-compose.yml (Tag Predictor)
`-- docker-compose.yml
```

Fig. 11 File structure of the project

The image shows the structure of the project with all the Microservices of all 3 use-cases. Each use-case has its own “docker-compose.yml” file. which allows it to run on separate hosts. However, all of the use-cases can also be run on a single host using the outermost “docker-compose.yml” file. Each service contains “Dockerfile” and other files required to run a particular microservice. The “Dockerfile” is responsible for all the services to run as a standalone Microservice.

Limitations

- Docker containers work on an isolated environment, therefore in order to establish communication between microservices, each container should either be on the same network or we need to develop REST api endpoints. This also raises a concern of establishing a secure connection and proves to be an additional overhead.

- Microservices architecture also introduces the problem of data redundancy, as each service handles data independently. It is possible that the same data could be needed by two different microservices, and this creates redundancy in data. At the same time, if we create a universal data store for all microservices to leverage data, then any alterations to database schema will affect all the services thereby negating the purpose of “independent” microservices.

- Scalability will also pose a problem as more microservices are added. The granular the application gets, the more complex it becomes.

Lessons Learned

- As all the services in the microservices act as a separate application, the second approach where we develop all the tests at once is best suitable. These services will be smaller and often will represent a single functionality of the entire application. Hence we found that writing all the tests for a particular service first and then developing the service worked best for us.

- Writing a system integration test at the beginning itself requires the visualisation of the architecture of the whole app in one's mind and it also means no additional requirements can be incorporated while the application is being developed.

- To first think about the design of the application, unit tests, integration of different modules before the actual implementation.

- TDD can help reduce the amount of extra code as we will be writing the code only for the test cases developed.

- Complex Applications with big data require extensive troubleshooting and TDD makes it easier to accomplish this.

- Small test cases sometimes may contain the actual code of the functionality that needs to be implemented and is being checked against itself. This will lead to defeat of the purpose of TDD approach.

- This approach can also lead to unrealistic simplification of the implementation which does not have much significance in delivering the final product.

Best Practises in TDD

1. Proper naming conventions were followed, so all the developers could understand what the test case is intended for.
2. Naming conventions in the test cases followed a similar fashion of the methods they were implemented to test.
3. Re-run all the tests whenever there is modification in the development.
4. No function dependencies were introduced between the tests
5. Multiple assert statements were avoided as they could lead to confusion where the test failed.
6. Simple codes were implemented in the passing the test

Conclusion

One of main advantages of Microservices is their capability of functioning independently. If any of the components fail, then it could be replaced and the application would continue to operate normally. It is also easier to understand the functionality of components as compared to traditional monolithic architecture. Communication between microservices happens smoothly despite the microservices being coded in different languages. Every microservice runs according to its own functionality and usually manages its own database. This lets the IT team deploy the code without having to wait until the entire application is ready. Despite their limitations, Microservices architecture's biggest pro is creating granular, focused teams that can develop independent functions faster, at a higher quality level.

Future Work

- Streaming the data directly to the Database.

  Currently the data is being downloaded by a Data Collector service and being sent to the Database after cleaning it, which increases the amount of space required to host a particular Data Collector service. To avoid this redundancy, the Data Collector service should be able to directly stream the data with proper testing.

- Deploying the built services onto different hosting platforms

  In its current state, the project acts as only a proof of concept and is virtually being deployed in a different host using Docker Compose. As future work, the services can be deployed and can be hosted on container-orchestration tools followed by testing the operations of microservices by replacing the individual containers in action.

References

1. "(PDF) Big Data-A Review on Analysing 3Vs - ResearchGate." 30 Oct. 2018, https://www.researchgate.net/publication/328607349_Big_Data-A_Review_on_Analysing_3Vs. Accessed 8 Dec. 2021.

2. S. Sagiroglu and D. Sinanc, "Big data: A review," 2013 International Conference on Collaboration Technologies and Systems (CTS), 2013, pp. 42-47, doi: 10.1109/CTS.2013.6567202.

3. Movie Tags Prediction Using Machine Learning Models https://medium.com/analytics-vidhya/movie-tag-s-prediction-using-machine-learning-models-d5fde119db6d

4. J. Cito, G. Schermann, J. E. Wittern, P. Leitner, S. Zumberi and H. C. Gall, "An Empirical Analysis of the Docker Container Ecosystem on GitHub," 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 323-333, doi: 10.1109/MSR.2017.67.

5. Kinsman, Timothy, Mairieli Wessel, Marco A. Gerosa, and Christoph Treude. "How Do Software Developers Use GitHub Actions to Automate Their Workflows?." *arXiv preprint arXiv:2103.12224* (2021).

6. Building a MovieLens Recommender System
https://github.com/topspinj/tmls-2020-recommender-workshop/blob/master/tutorial.ipynb

7. F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4: 19:1–19:19. <https://doi.org/10.1145/2827872>